

Week 5

1. Refactoring / Code “Smells”

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2800 – Software Development [2026W]

Monday, February 2nd, 2026 | 14:30 – 15:50 | ER 1118



University of Windsor

Agenda



Software Engineering:

- **Refactoring**
 - **Code “Smells”**
-

Part I

Refactoring



○ The problems with coding



● Improving the design of existing code

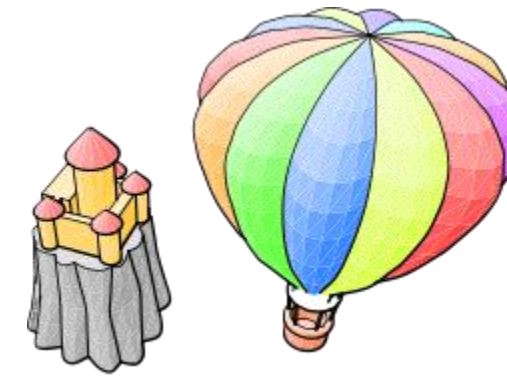
- Problems observed in OO programs
 - Poorly designed programs written by inexperienced programmers
 - Inefficient applications
 - Hard to maintain applications
 - Hard to extend applications
- Difficult to work with these inherited “non-optimal” applications
- Thus, what to do? Refactoring! Refactoring and Refactoring ...



Refactoring



Refactoring



- Term originated with developers with Smalltalk
 - Expert OO programmers employed techniques to improve the structural integrity and performance of applications
 - Refactoring was used to develop efficient and flexible frameworks
 - Martin Fowler produced a catalogue of these techniques in a 1999 text (with help from Kent Beck and others)
- The idea is to change your code a small step at a time and thus reduce the risk of evolving your design
- Note: refactoring techniques were used well before the term “refactoring” was coined and before OO programming

Refactoring

- Refactoring is the process of changing a software system in such a way that
 - it does not alter external behaviour of the code yet improves its internal structure
- Refactoring is the disciplined way of cleaning up code
 - that minimizes the chances of introducing bug
- When you refactor you are improving the design of the code after it has been written
 - design occurs continuously during development



● Refactoring as Noun and Verb

- Refactoring (Noun)
 - A change is made to the internal structure of software to make it **easier to understand and cheaper to modify** without changing the observable behaviour of the software
- Refactor (Verb)
 - To restructure software by applying a series of refactorings without changing the observable behaviour of the software

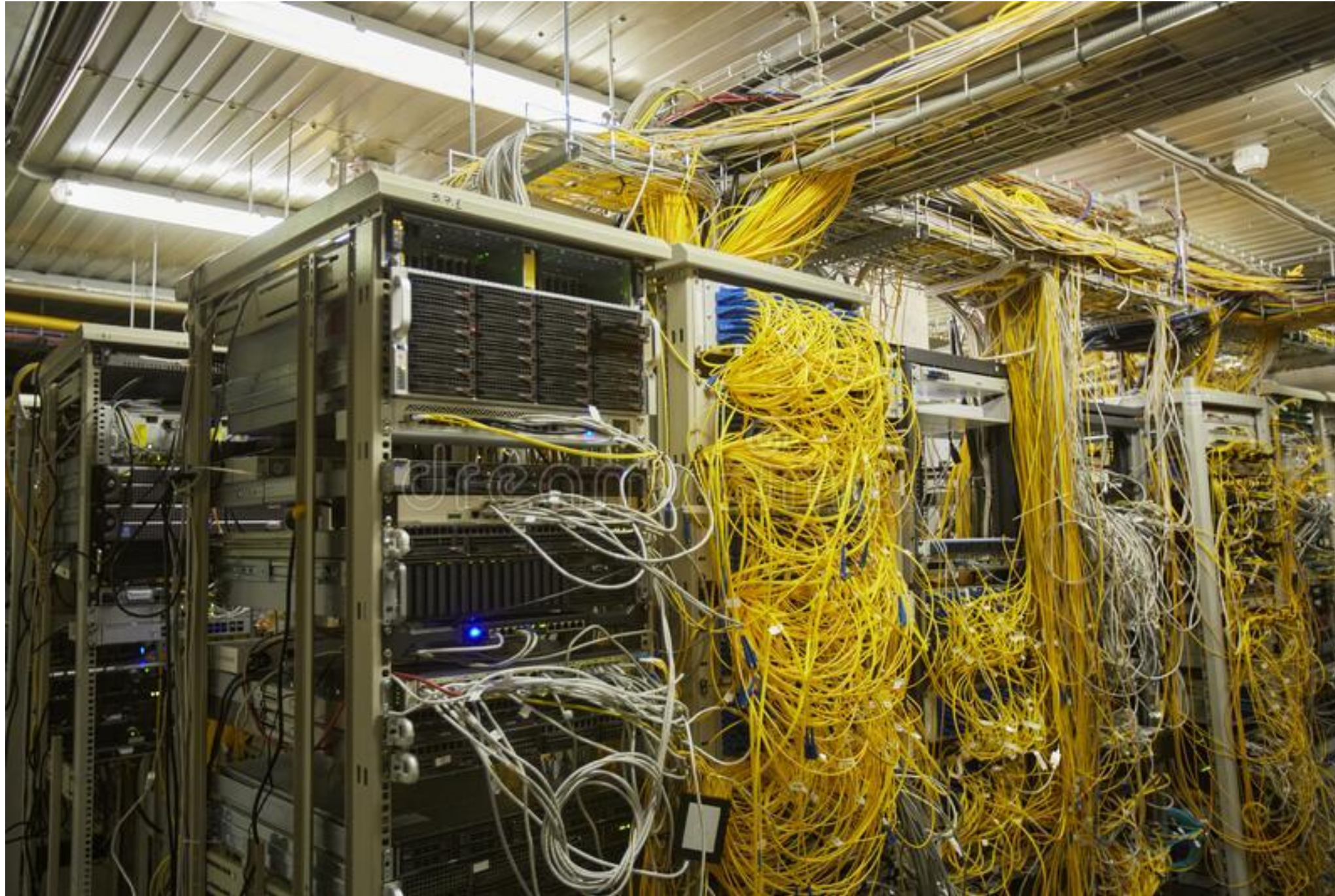


Testing and Refactoring

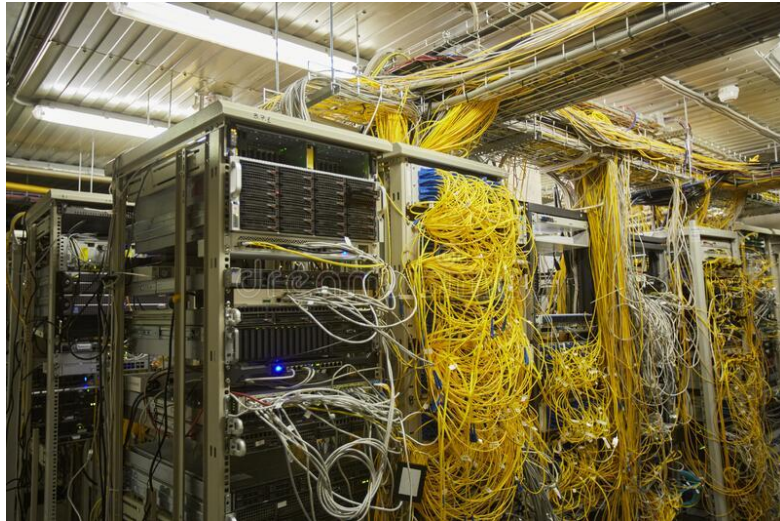
- Testing is an important part of refactoring
 - Refactoring changes the program in a small step
 - If you make a mistake, it is easy to find the bug
- Testing Advice
 - Before you start refactoring, check that you have a solid suite of tests
 - Make sure that all tests are fully automatic and that they check their own results (“tests must be self-testing”)
 - Run your test frequently
 - Tool support
 - JUnit testing framework



Why Refactor?



Why Refactor?



Why Refactor?

- Refactoring improves the design of the software
 - Lehman's Law of Software Evolution II-**Increased Complexity**
 - As a program is evolved, its complexity increases unless work is done to maintain or reduce it
 - regular refactoring helps code retain its shape!
- Refactoring makes software easier to understand
 - Refactoring will help structure the program to better communicate its purpose
- Refactoring helps you find bugs
 - Refactoring will help you to understand the code in more detail. The deeper understanding of the code will help you find more bugs
- Refactoring helps you program faster
 - Good design promotes rapid software development



When to Refactor?

- Refactor when you find you have to add a feature to a program
 - First refactor the program to make it easy to add the feature and then add the feature – pre-factoring!
- Refactor when you feel the need to write a comment
 - first try to refactor the code so that no comments become superfluous
- The rule of three
 - The first time you do it, just do it. The second time you do something similar, wince and do it. The third time you do something similar, Refactor!
- Refactor when you need to fix a bug
 - Refactoring helps to make the code more understandable and helps you better understand the code. So, refactor first and then fix the bug.



Refactoring Examples



Examples of Refactoring

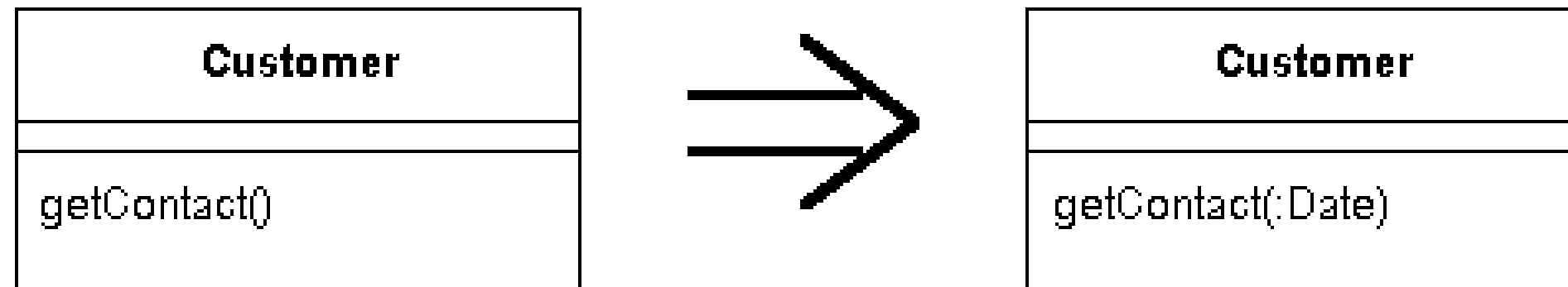
- Add Parameter
- Consolidate Conditional Expression
- Extract Class
- Hide Method
- Inline Method
- Introduce Assertion
- Move Method
- Parameterize Method
- Rename Method
- Replace Array with Object ...

Could any of these refactoring change the external behaviour of a program??



Add Parameter

- *A method needs more information from its caller.*
 - **Add a parameter for an object that can pass on this information.**



Add Parameter: Mechanics

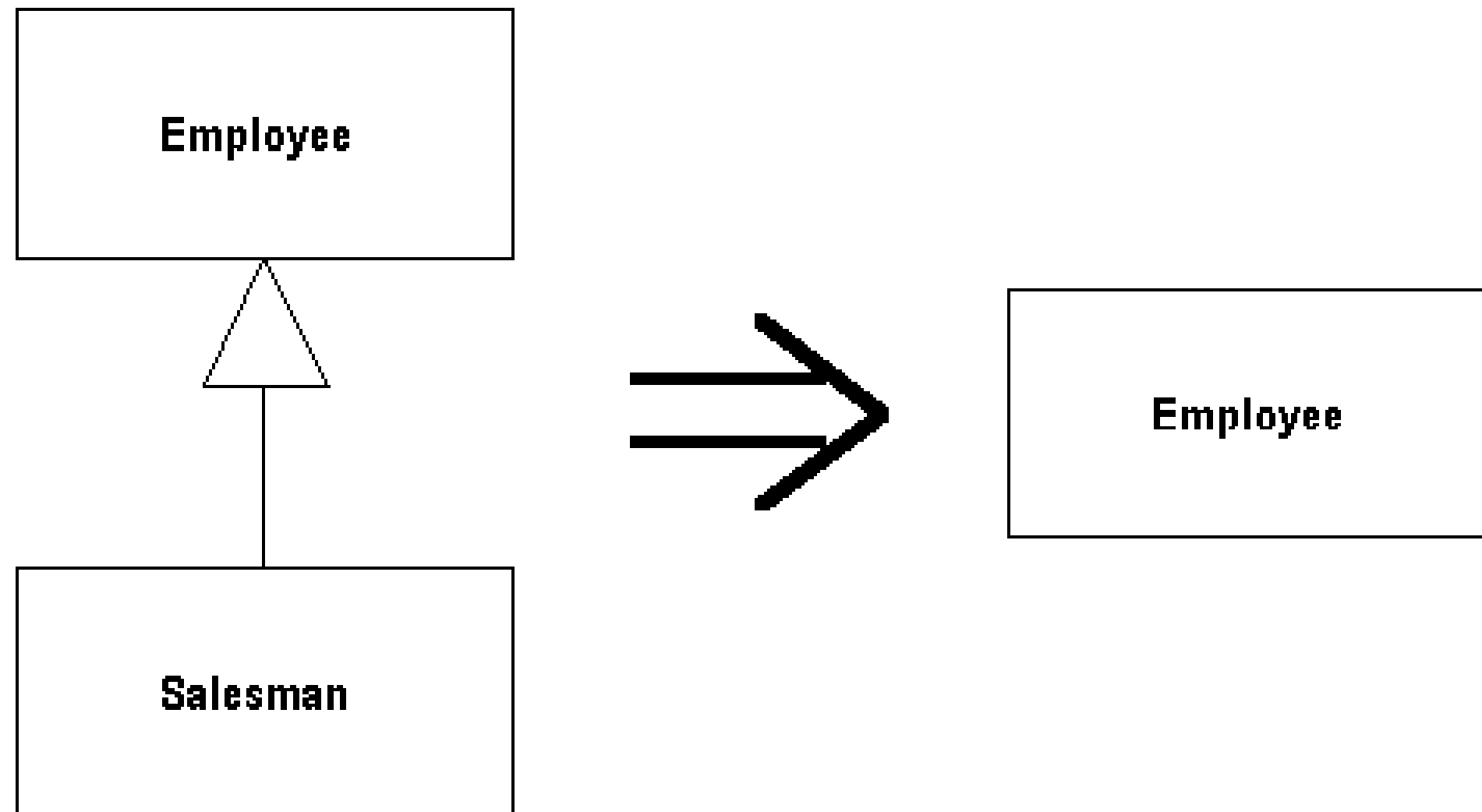
- **Mechanics**

- Check to see whether the method signature is implemented by a superclass or subclass. If it is, perform these steps for each implementation:
 - Declare a new method with the added parameter. Copy the old body of code over to the new name and make any alterations to fit.
 - Compile.
 - Change the body of the old method so that it calls the new one.
 - Compile and test.
 - Find all references to the old method and change them to refer to the new one.
 - Compile and test after each change.
 - Remove the old method. (If the old method is part of the published interface, you cannot remove it: leave it in place and mark it as deprecated.)
 - Compile and test.



Collapse Hierarchy

- *A superclass and subclass are not very different.*
 - **Merge them together.**



Consolidate Conditional Expression

- *You have a sequence of conditional tests with the same result.*
 - **Combine them into a single conditional expression and extract it.**

```
double disabilityAmount() {  
    if (_seniority < 2) return 0;  
    if (_monthsDisabled > 12) return 0;  
    if (!_isPartTime) return 0;  
    // compute the disability amount
```



```
double disabilityAmount() {  
    if (isNotEligibleForDisability()) return 0;  
    // compute the disability amount
```

Consolidate Duplicate Conditional Fragments

- *The same fragment of code is in all branches of a conditional expression.*
 - **Move it outside of the expression**

```
if (isSpecialDeal()) {  
    total = price * 0.95;  
    send();  
}  
else {  
    total = price * 0.98;  
    send();  
}
```



```
if (isSpecialDeal())  
    total = price * 0.95;  
else  
    total = price * 0.98;  
send();
```

Decompose Conditional

- *You have a complicated conditional (if-then-else) statement.*
 - Extract methods from the condition, then part, and else parts.

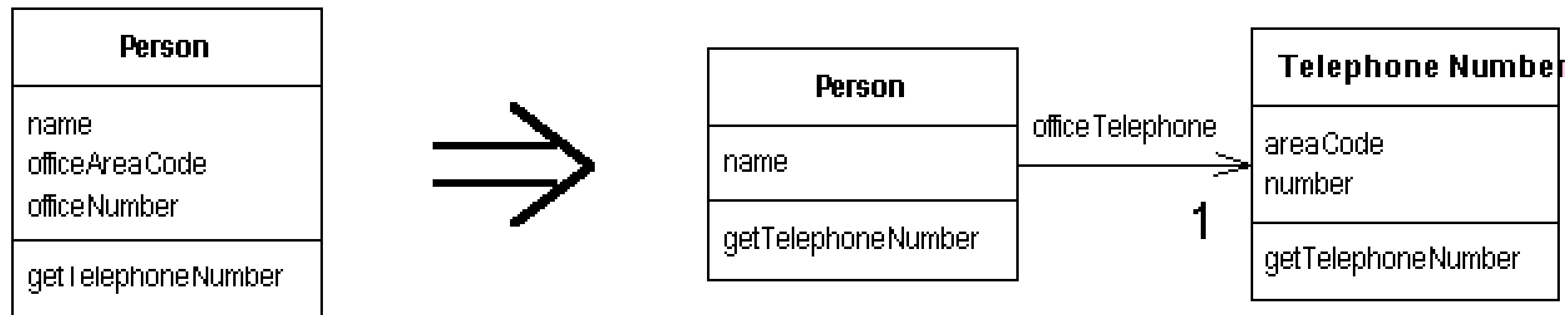
```
if (date.before (SUMMER_START) || date.after(SUMMER_END))  
    charge = quantity * _winterRate + _winterServiceCharge;  
else charge = quantity * _summerRate;
```



```
if (notSummer(date))  
    charge = winterCharge(quantity);  
else charge = summerCharge (quantity);
```

Extract Class

- *You have one class doing work that should be done by two.*
 - Create a new class and move the relevant fields and methods from the old class into the new class.



Extract Method

- *You have a code fragment that can be grouped together.*
 - **Turn the fragment into a method whose name explains the purpose of the method.**

```
void printOwing() {  
    printBanner();
```

```
    //print details  
    System.out.println ("name:      " + _name);  
    System.out.println ("amount    " + getOutstanding());  
}
```

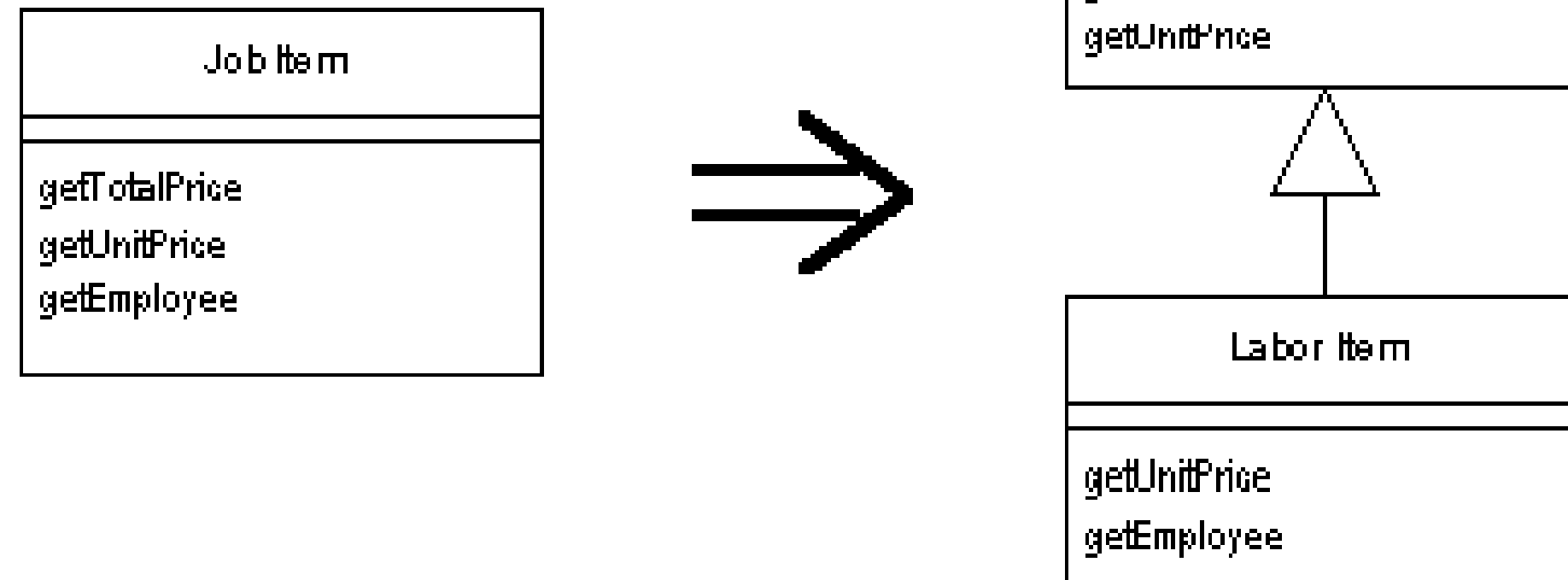


```
void printOwing() {  
    printBanner();  
    printDetails(getOutstanding());  
}
```

```
void printDetails (double outstanding) {  
    System.out.println ("name:      " + _name);  
    System.out.println ("amount    " + outstanding);  
}
```

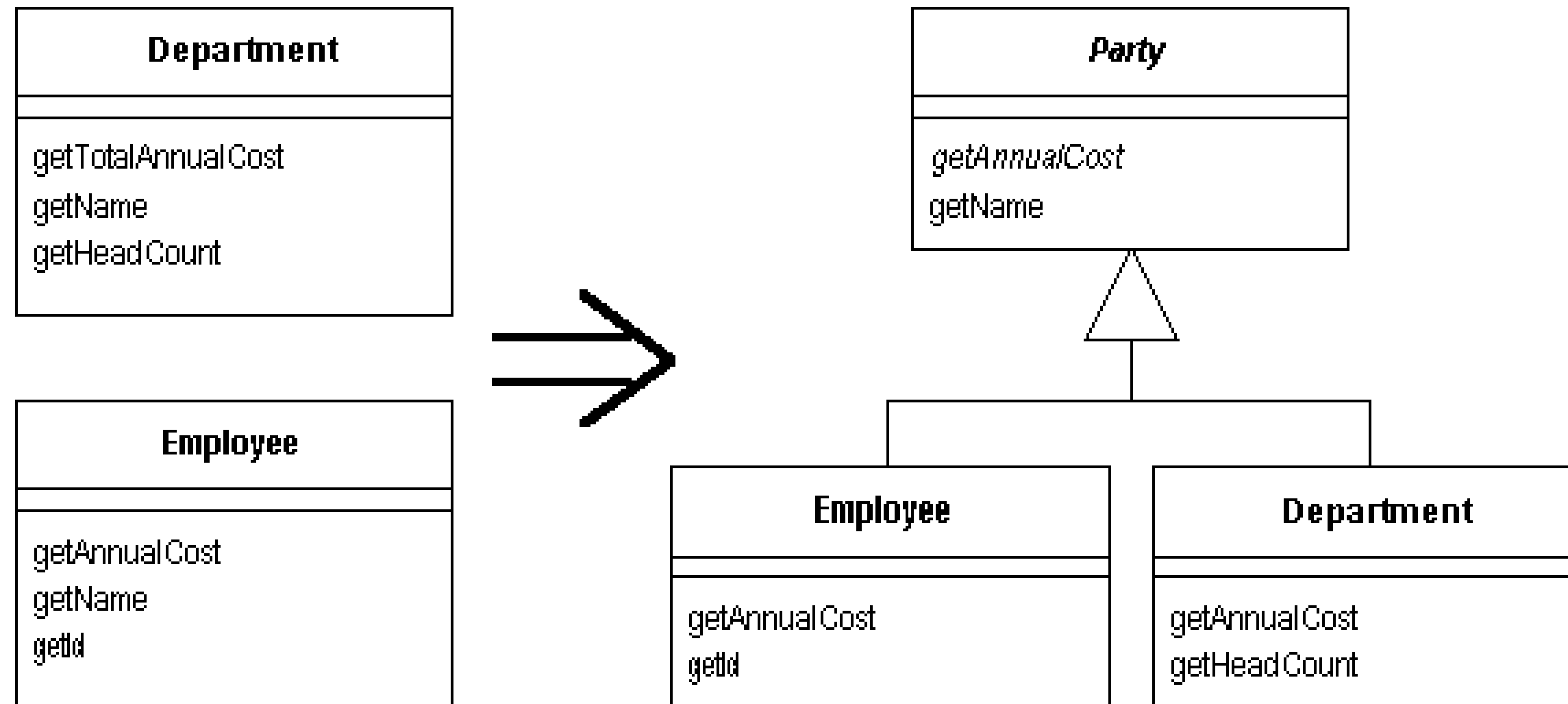
Extract Subclass

- *A class has features that are used only in some instances.*
 - **Create a subclass for that subset of features.**



Extract Superclass

- *You have two classes with similar features.*
 - Create a superclass and move the common features to the superclass.



Introduce Explaining Variable

- *You have a complicated expression.*
 - Put the result of the expression, or parts of the expression, in a temporary variable with a name that explains the purpose.

```
if ( (platform.toUpperCase().indexOf("MAC") > -1) &&  
      (browser.toUpperCase().indexOf("IE") > -1) &&  
      wasInitialized() && resize > 0 )  
{  
    // do something  
}
```

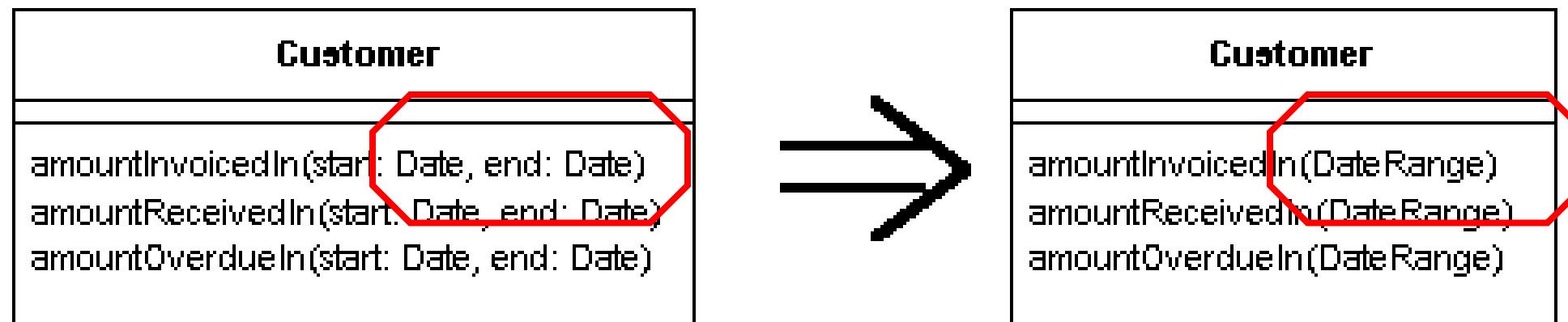


```
final boolean isMacOs      = platform.toUpperCase().indexOf("MAC") > -1;  
final boolean isIEBrowser  = browser.toUpperCase().indexOf("IE") > -1;  
final boolean wasResized   = resize > 0;
```

```
if (isMacOs && isIEBrowser && wasInitialized() && wasResized)  
{  
    // do something  
}
```

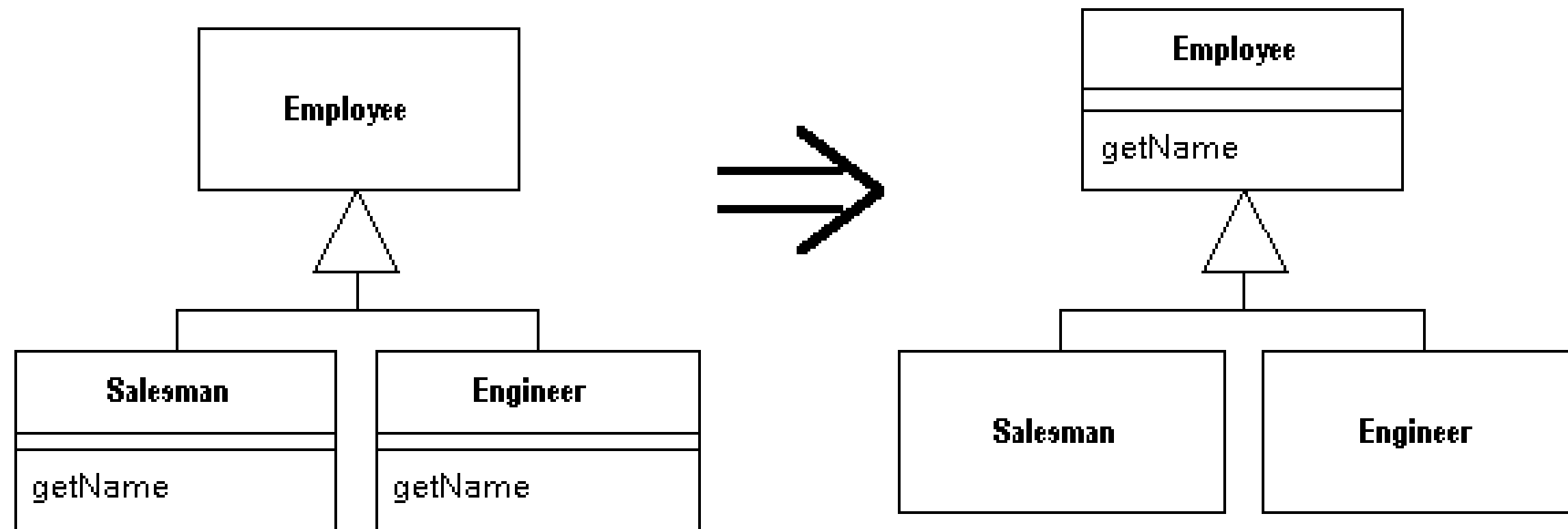
Introduce Parameter Object

- *You have a group of parameters that naturally go together.*
 - **Replace them with an object.**



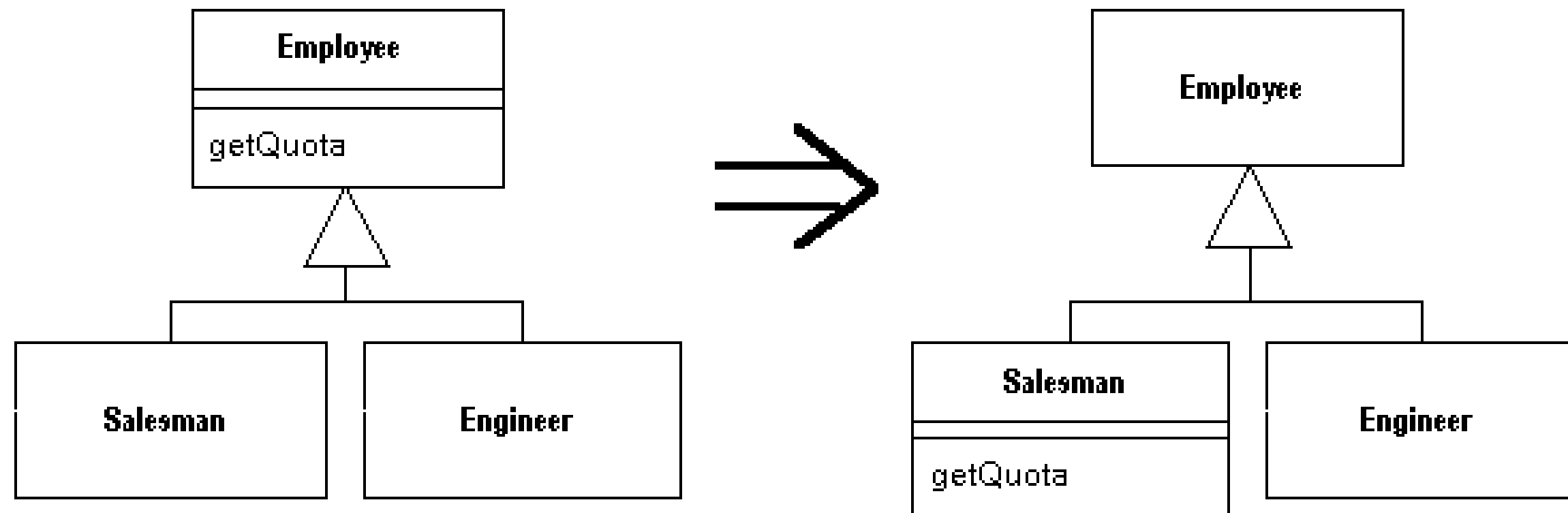
Pull Up Method

- *You have methods with identical results on subclasses.*
 - **Move them to the superclass.**



Push Down Method

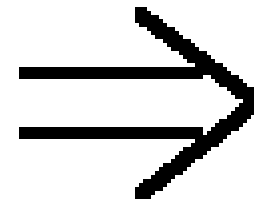
- *Behavior on a superclass is relevant only for some of its subclasses.*
 - **Move it to those subclasses.**



Reduce Scope of Variable

- *You have a local variable declared in a scope that is larger than where it is used*
 - **Reduce the scope of the variable so that it is only visible in the scope where it is used**

```
void foo()
{
    int i = 7;
    // i is not used here
    if (someCondition)
    {
        // i is used only within this block
    }
    // i is not used here
}
```

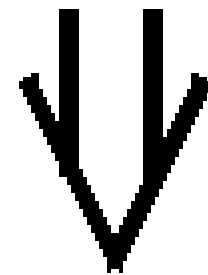


```
void foo()
{
    // i can not be used here
    if (someCondition)
    {
        int i = 7;
        // i is used only within this block
    }
    // i can not be used here
}
```

Remove Assignments to Parameters

- *The code assigns to a parameter.*
 - **Use a temporary variable instead.**

```
int discount (int inputVal, int quantity, int yearToDate) {  
    if (inputVal > 50) inputVal -= 2;  
}
```



```
int discount (int inputVal, int quantity, int yearToDate) {  
    int result = inputVal;  
    if (inputVal > 50) result -= 2;  
}
```

● Remove Double Negative

- *You have a double negative conditional.*
 - **Make it a single positive conditional**

```
if ( !item.isNotFound() )
```



```
if ( item.isFound() )
```


Replace Array with Object

- *You have an array in which certain elements mean different things.*
 - Replace the array with an object that has a field for each element.

```
String[] row = new String[3];  
row [0] = "Liverpool";  
row [1] = "15";
```



```
Performance row = new Performance();  
row.setName("Liverpool");  
row.setWins("15");
```

● Replace Assignment with Initialization

- *You have code that first declares a variable and then assigns a value to it*
 - **Make it into a direct initialization instead**

```
void foo() {  
    int i;  
    // ....  
    i = 7;  
}
```



```
void foo() {  
    // ...  
    int i = 7;  
}
```

● Replace Magic Number with Symbolic Constant

- *You have a literal number with a particular meaning.*
 - **Create a constant, name it after the meaning, and replace the number with it.**

```
double potentialEnergy(double mass, double height) {  
    return mass * height * 9.81;  
}
```

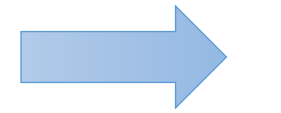


```
double potentialEnergy(double mass, double height) {  
    return mass * GRAVITATIONAL_CONSTANT * height;  
}  
static final double GRAVITATIONAL_CONSTANT = 9.81;
```

Replace Temp with Query

- *You are using a temporary variable to hold the result of an expression.*
 - **Extract the expression into a method. Replace all references to the temp with the expression. The new method can then be used in other methods.**

```
double basePrice = _quantity * _itemPrice;  
if (basePrice > 1000)  
    return basePrice * 0.95;  
else  
    return basePrice * 0.98;
```



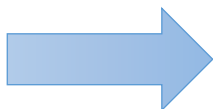
...

```
if (basePrice() > 1000)  
    return basePrice() * 0.95;  
else  
    return basePrice() * 0.98;  
  
double basePrice() {  
    return _quantity * _itemPrice;  
}
```

Split Loop

- *You have a loop that is doing two things*
 - **Duplicate the loop**

```
void printValues() {  
    double averageAge = 0;  
    double totalSalary = 0;  
    for (int i = 0; i < people.length; i++) {  
        averageAge += people[i].age;  
        totalSalary += people[i].salary;  
    }  
    averageAge = averageAge / people.length;  
    System.out.println(averageAge);  
    System.out.println(totalSalary);  
}
```



```
void printValues() {  
    double totalSalary = 0;  
    for (int i = 0; i < people.length; i++) {  
        totalSalary += people[i].salary;  
    }  
  
    double averageAge = 0;  
    for (int i = 0; i < people.length; i++) {  
        averageAge += people[i].age;  
    }  
    averageAge = averageAge / people.length;  
  
    System.out.println(averageAge);  
    System.out.println(totalSalary);  
}
```

- *Doing the loop twice-may be bad*
 - **But for sure it increases the understandability!**

Split Loop

- **Mechanics**

- Copy the loop and remove the differing pieces from each loop
- Compile and test
- Reorganize the lines to group the loop with related code from outside the loop
- Compile and test.
- Consider applying Extract Method or Replace Temp with Query on each loop



Split Loop: Example

- Start with this code
 - The loop is calculating both the total salary and the average age
 - Let's split the loop

```
private Person [] people;

void printValues() {
    double averageAge = 0;
    double totalSalary = 0;
    for (int i = 0; i < people.length; i++) {
        averageAge += people[i].age;
        totalSalary += people[i].salary;
    }
    averageAge = averageAge / people.length;
    System.out.println(averageAge);
    System.out.println(totalSalary);
}
```



Split Loop: Example

- Split the loop

- The first move is to copy the loop and remove from each leg one part of the calculation
- Compile and test



```
void printValues() {  
    double averageAge = 0;  
    double totalSalary = 0;  
    for (int i = 0; i < people.length; i++) {  
        totalSalary += people[i].salary;  
    }  
    for (int i = 0; i < people.length; i++) {  
        averageAge += people[i].age;  
    }  
    averageAge = averageAge / people.length;  
    System.out.println(averageAge);  
    System.out.println(totalSalary);  
}
```

Split Loop: Example

- Then we can reorganize the text to group things together.



```
void printValues() {  
    double totalSalary = 0;  
    for (int i = 0; i < people.length; i++) {  
        totalSalary += people[i].salary;  
    }  
  
    double averageAge = 0;  
    for (int i = 0; i < people.length; i++) {  
        averageAge += people[i].age;  
    }  
    averageAge = averageAge / people.length;  
  
    System.out.println(averageAge);  
    System.out.println(totalSalary);  
}
```

- Duplicated loops and duplication is the number 1 bad smell!
 - But it is for good things ...

Split Loop: Example

- Apply **Replace Temp with Query** to each loop

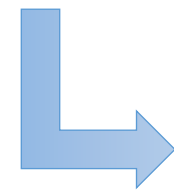


```
void printValues() {  
    System.out.println(averageAge());  
    System.out.println(totalSalary());  
}  
  
private double averageAge() {  
    double result = 0;  
    for (int i = 0; i < people.length; i++) {  
        result += people[i].age;  
    }  
    return result / people.length;  
}  
  
private double totalSalary() {  
    double result = 0;  
    for (int i = 0; i < people.length; i++) {  
        result += people[i].salary;  
    }  
    return result;  
}
```

Substitute Algorithm

- *You want to replace an algorithm with one that is clearer.*
 - **Replace the body of the method with the new algorithm.**

```
String foundPerson(String[] people){
    for (int i = 0; i < people.length; i++) {
        if (people[i].equals ("Don")){
            return "Don";
        }
        if (people[i].equals ("John")){
            return "John";
        }
        if (people[i].equals ("Kent")){
            return "Kent";
        }
    }
    return "";
}
```



```
String foundPerson(String[] people){
    List candidates = Arrays.asList(new String[] {"Don", "John", "Kent"});
    for (int i=0; i<people.length; i++)
        if (candidates.contains(people[i]))
            return people[i];
    return "";
}
```


Part II

Bad code “smells”

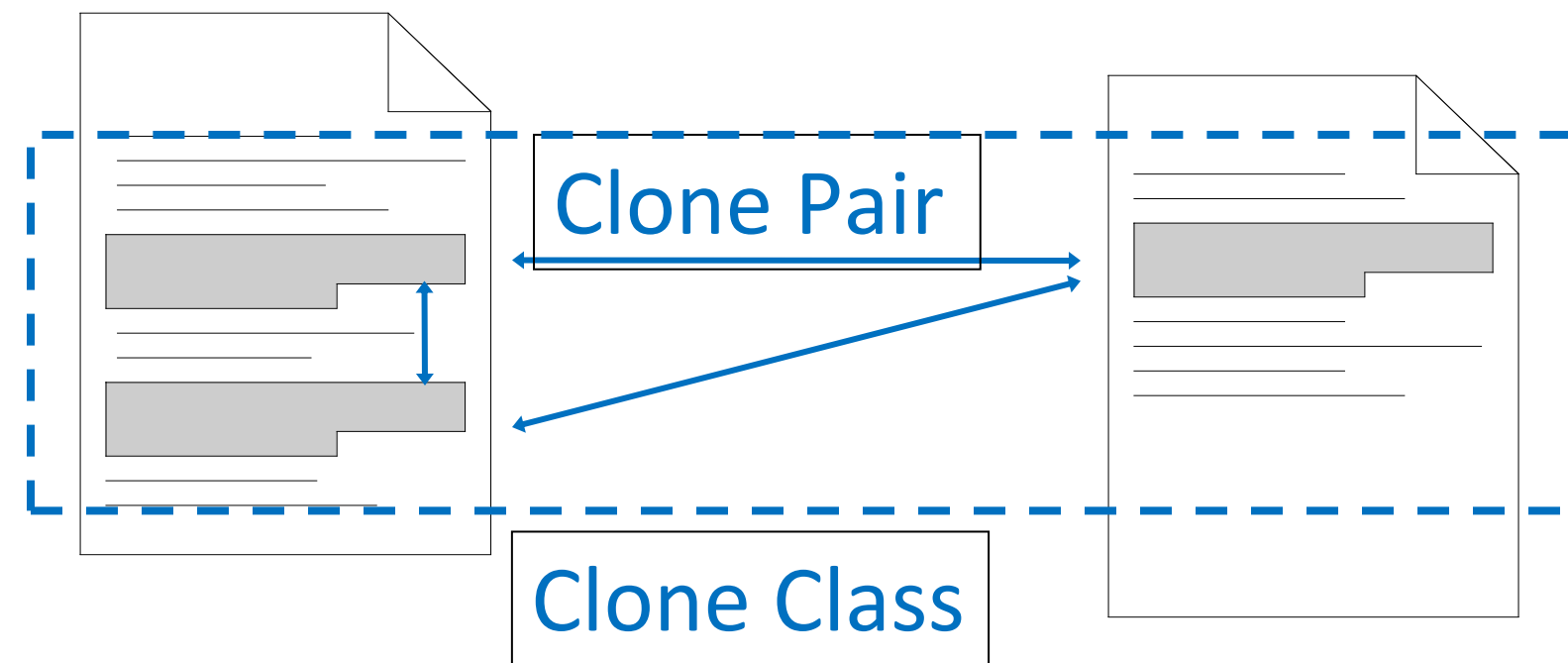


○ Duplicate code → No 1 “smell”



Software Clone or Code Clone?

- A code fragment which has identical or similar code fragment (s) in source code



A grayscale background image featuring several Star Wars Clone Troopers. In the center, a trooper stands prominently, flanked by others. In the foreground, two trooper helmets are visible. The image is semi-transparent, allowing the text and footer to be seen over it.

Code clones

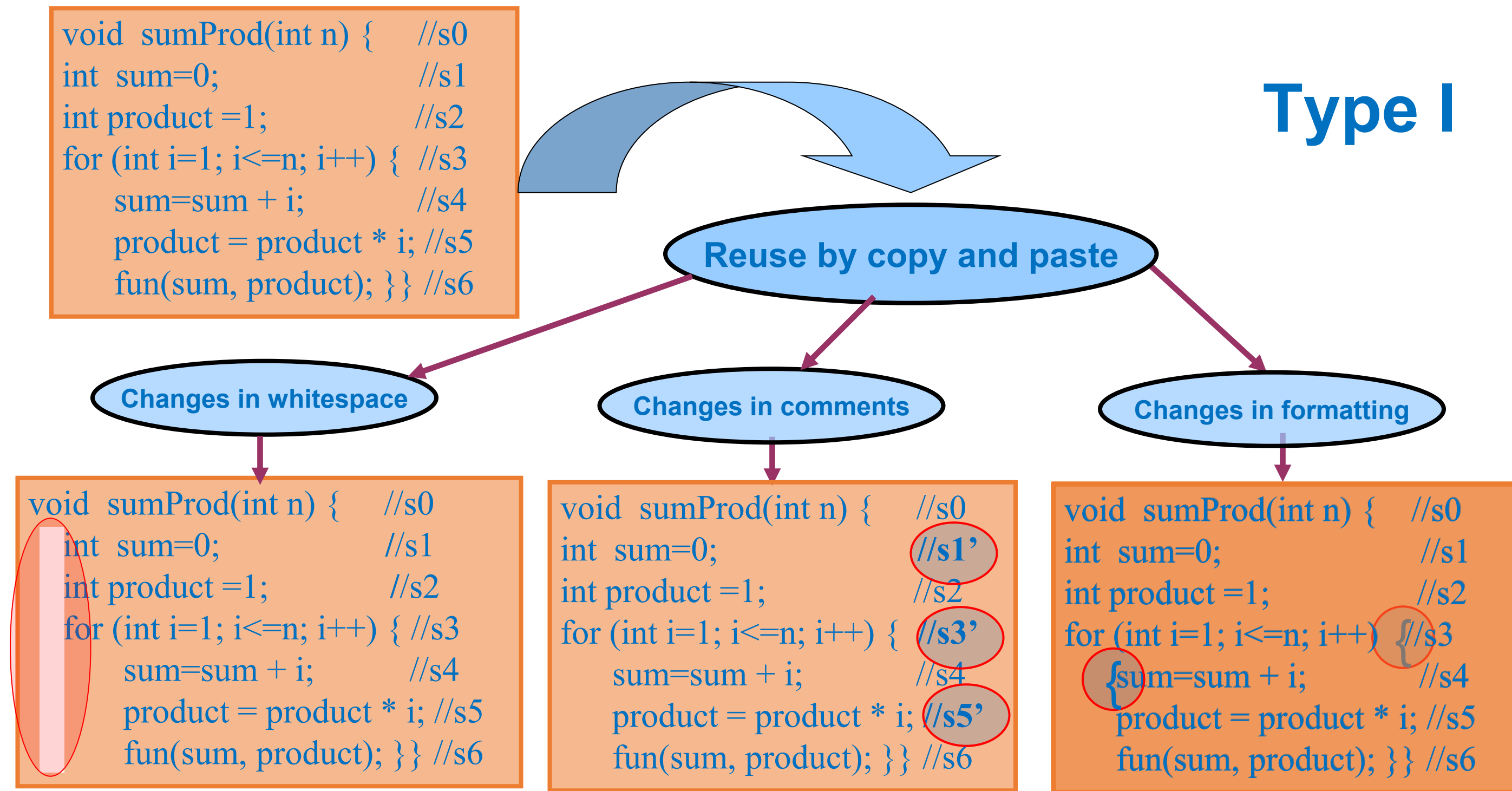


University of Windsor

Exact Software Clones

Changes in layout and formatting

Type I



Near-Miss Software Clone

Renaming Identifiers and Literal Values

```
void sumProd(int n) { //s0
int sum=0; //s1
int product=1; //s2
for (int i=1; i<=n; i++) { //s3
    sum=sum + i; //s4
    product = product * i; //s5
    fun(sum, product); } //s6
```

sumProd => addTimes

sum => add

product => times

0=>0.0

1=>1.0

int=>double

Reuse by copy and paste

Renaming of identifiers

```
void addTimes(int n) { //s0
int add=0; //s1
int times =1; //s2
for (int i=1; i<=n; i++) { //s3
    add=add + i; //s4
    times = times * i; //s5
    fun(add, times); } //s6
```

Renaming of Literals and Types

```
void sumProd(int n) { //s0
double sum=0.0; //s1
double product=1.0; //s2
for (int i=1; i<=n; i++) { //s3
    sum=sum + i; //s4
    product = product * i; //s5
    fun(sum, product); } //s6
```

Type II

Near-Miss Software Clone

Statements added/deleted/modified in copied fragments

```
void sumProd(int n) { //s0
int sum=0;           //s1
int product =1;      //s2
for (int i=1; i<=n; i++) { //s3
    sum=sum + i;      //s4
    product = product * i; //s5
    fun(sum, product); } } //s6
```

Reuse by copy and paste

Type III

Modification of lines

```
void sumProd(int n) { //s0
int sum=0;           //s1
int product =1;      //s2
for (int i=1; i<=n; i++) { //s3
    if (i % 2 == 0) sum+= i; //s4m
    product = product * i; //s5
    fun(sum, product); } } //s6
```

Addition of new of lines

```
void sumProd(int n) { //s0
int sum=0;           //s1
int product =1;      //s2
for (int i=1; i<=n; i++) //s3
    { div=sum/2 //s3b
    sum=sum + i;      //s4
    product = product * i; //s5
    fun(sum, product); } } //s6
```

Deletions of lines

```
void sumProd(int n) { //s0
int sum=0;           //s1
int product =1;      //s2
for (int i=1; i<=n; i++) { //s3
    sum=sum + i;      //s4
    //s5 line deleted
    fun(sum, product); } } //s6
```


Semantic Software Clone

Statements reordering/control replacements

```
void sumProd(int n) { //s0
int sum=0;           //s1
int product=1;       //s2
for (int i=1; i<=n; i++) { //s3
    sum=sum + i;      //s4
    product = product * i; //s5
    fun(sum, product); } //s6
```

Reuse by copy and paste

Reordering of Statements

```
void sumProd(int n) { //s0
int product=1;      //s2
int sum=0;          //s1
for (int i=1; i<=n; i++) { //s3
    sum=sum + i;      //s4
    product = product * i; //s5
    fun(sum, product); } //s6
```

Type IV

Control Replacements

```
void sumProd(int n) { //s0
int sum=0;           //s1
int product=1;       //s2
int i = 0;          //s7
while (i<=n) {       //s3'
    sum=sum + i;      //s4
    product = product * i; //s5
    fun(sum, product); //s6
i=i + 1; }          //s8
```


Are there Clones in Software?

- Significant amount of cloned code
- Varies depending on the domain and origin.
 - 7%-23% Baker WCRE'05
 - 12.7% Baxter et al. ICSM'98
 - 5%-20% Mayrand et al. ICSM'96
 - 10%-15% Kapser and Godfrey JSME'06
- Sometimes, even about 50%, OO COBOL system, Ducasse et al. JSME'06

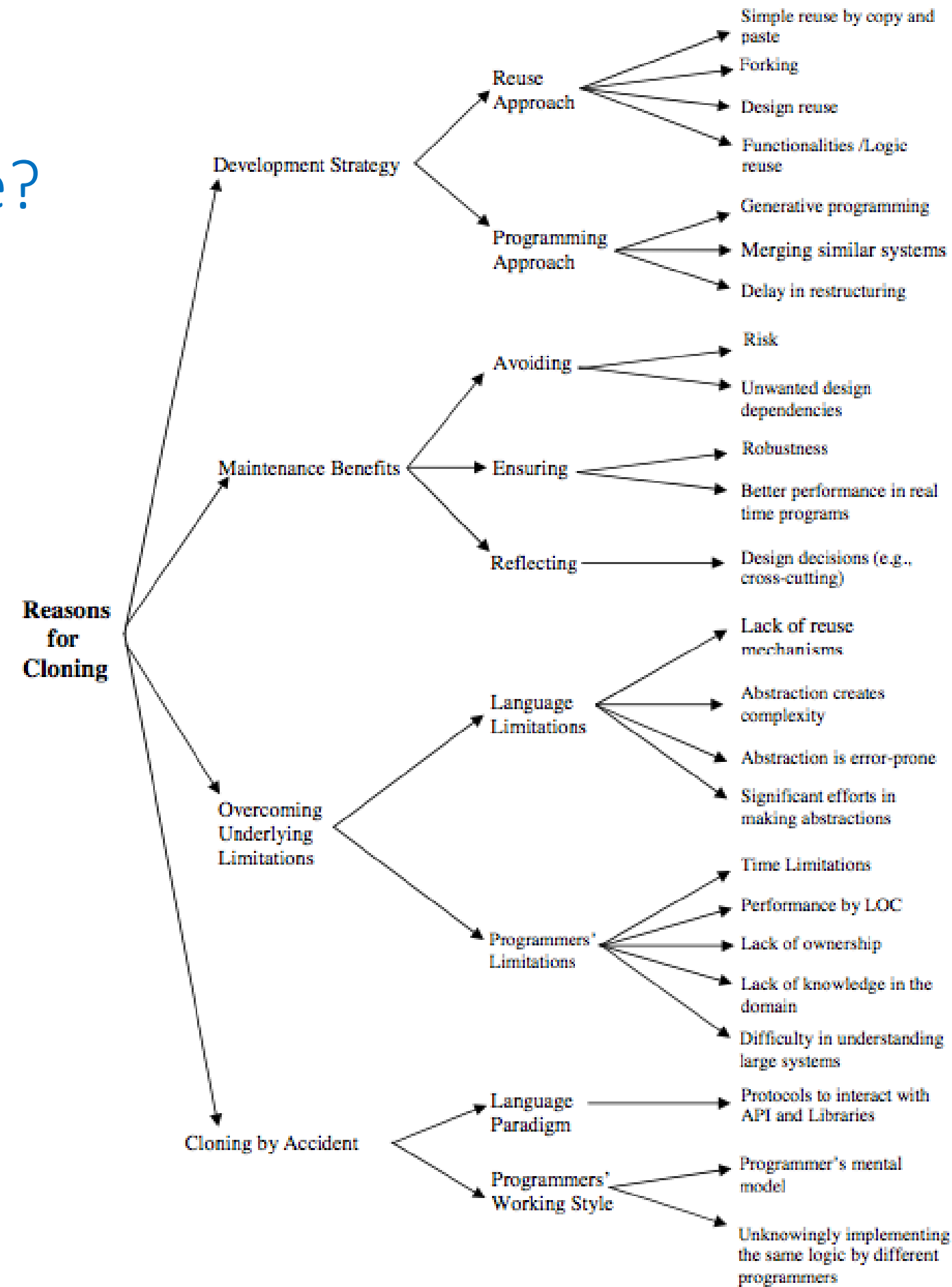


● Why are there Clones in Software?

- • Copy & paste is a common practice in software development
- Time limit assigned to programmers
- Risk in developing new code
- Language limitations
- And many more ... (see next slide for a tree diagram)



Why are there Clones in Software?



[Reference: Roy and Cordy's technical report \(pages 3 – 8\)](#)

Do Code Clones Matter?

- Are they Harmful? Yes, Juergens et al. ICSE'09
“Inconsistent changes to code duplicates are frequent and lead to severe unexpected behavior”
- Number 1 Bad Smell in the community Beck and Fowler's “Stink Parade of Bad Smells”
- Increased cognitive effort
 - Difficult to understand the differences between the cloned code
- Code bloat
 - unnecessarily long code



Clone Detection Process

*“Software entities are more complex for their size than perhaps any other human construct because **no two parts are alike** (at least above the statement level).*

***If they are, we make the two similar parts into a subroutine-** open or closed.*

In this respect, software systems differ profoundly from computers, buildings, or automobiles, where repeated elements abound.”

Frederick P. Brooks, Jr:
No Silver Bullet: Essence and Accidents of Software Engineering

Slide courtesy of: Rainer Koschke



Existing Clone Detection Methods

- Because of the importance, many methods proposed:
 - **Text-based:** Duploc [Ducasse et al. ICSM'99], NICAD [Roy and Cordy, ICPC'08]
 - **Token-based:** Dup [Baker, WCRE'95], CCFinder [Kamiya et al., TSE'02], CP-Miner [Li et al., TSE'06]
 - **Tree-Based:** CloneDr [Baxter et al. ICSM'98], Asta [Evans et al. WCRE'07], Deckard [Jiang et al. ICSE'07], cpdetector [Falke et al. ESE'08]
 - **Metrics-based:** Kontogiannis [WCRE'97], Mayrand et al. ICSM'96
 - **Graph-based:** Gabel et al. ICSE'08, Komondoor and Horwitz SAS'01, Dublix [Krinke WCRE'01]

See more on ...

This is the online catalog of refactorings, to support my book Refactoring 2nd Edition.

This catalog of refactorings includes those refactorings described in my original book on Refactoring, together with the Ruby Edition.

• <http://www.refactoring.com/catalog/>

Using the Catalog ▶

Tags

- ☐ basic
- ☐ encapsulation
- ☐ moving-features
- ☐ organizing-data
- ☐ simplify-conditional-logic
- ☐ refactoring-apis
- ☐ dealing-with-inheritance
- ☐ collections
- ☐ delegation
- ☐ errors
- ☐ extract
- ☐ parameters
- ☐ fragments
- ☐ grouping-function
- ☐ immutability
- ☐ inline
- ☐ remove

Change Function Declaration

Add Parameter • Change Signature • Remove Parameter • Rename Function • Rename Method

Change Reference to Value

Change Value to Reference

Collapse Hierarchy

Combine Functions into Class

Combine Functions into Transform

Consolidate Conditional Expression

Remove Dead Code

Remove Flag Argument

Replace Parameter with Explicit Methods

Remove Middle Man

Remove Setting Method

Remove Subclass

Replace Subclass with Fields

Rename Field

Rename Variable

Code “Smell” and Common Refactoring

- **Smell 1: Duplicated Code**

- Extract Method, Extract Class, Pull Up Method, Form Template Method

- **Smell 2: Alternative Classes with different interfaces**

- Rename Method, Move Method

- **Smell 3: Long Method**

- Extract Method, Replace Temp with Query, Replace Method with Method Object, Decompose Conditional

- **Smell 4: Large Class**

- Extract Class, Extract Subclass, Extract Interface, Replace Data Value with Object

- **Smell 5: Long Parameter List**

- Replace Parameter with Method, Introduce Parameter Object, Preserve Whole Object



Code Smell and Common Refactoring

- **Smell 6: Divergent Change**

(one class is commonly changed in different ways for different reasons)

- Extract Class

- **Smell 7: Shotgun Surgery**

(a lot of little changes to a lot of different classes)

- Move Method, Move Field, Inline Class

- **Smell 8: Switch Statements**

- Replace Conditional with Polymorphism, Replace Type Code with Subclasses, Replace Type Code with State/Strategy, Replace Parameter with Explicit Methods, Introduce Null Object

- **Smell 9: Parallel Inheritance Hierarchies**

(Every time you make a subclass of one class you also make a subclass of another one)

- Move Method, Move Field



Part III

What's Next



University of Windsor

Next Lecture



University of Windsor

● Next lecture

- • Next lecture includes:
 - Java Concurrency



Thank you!
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor